

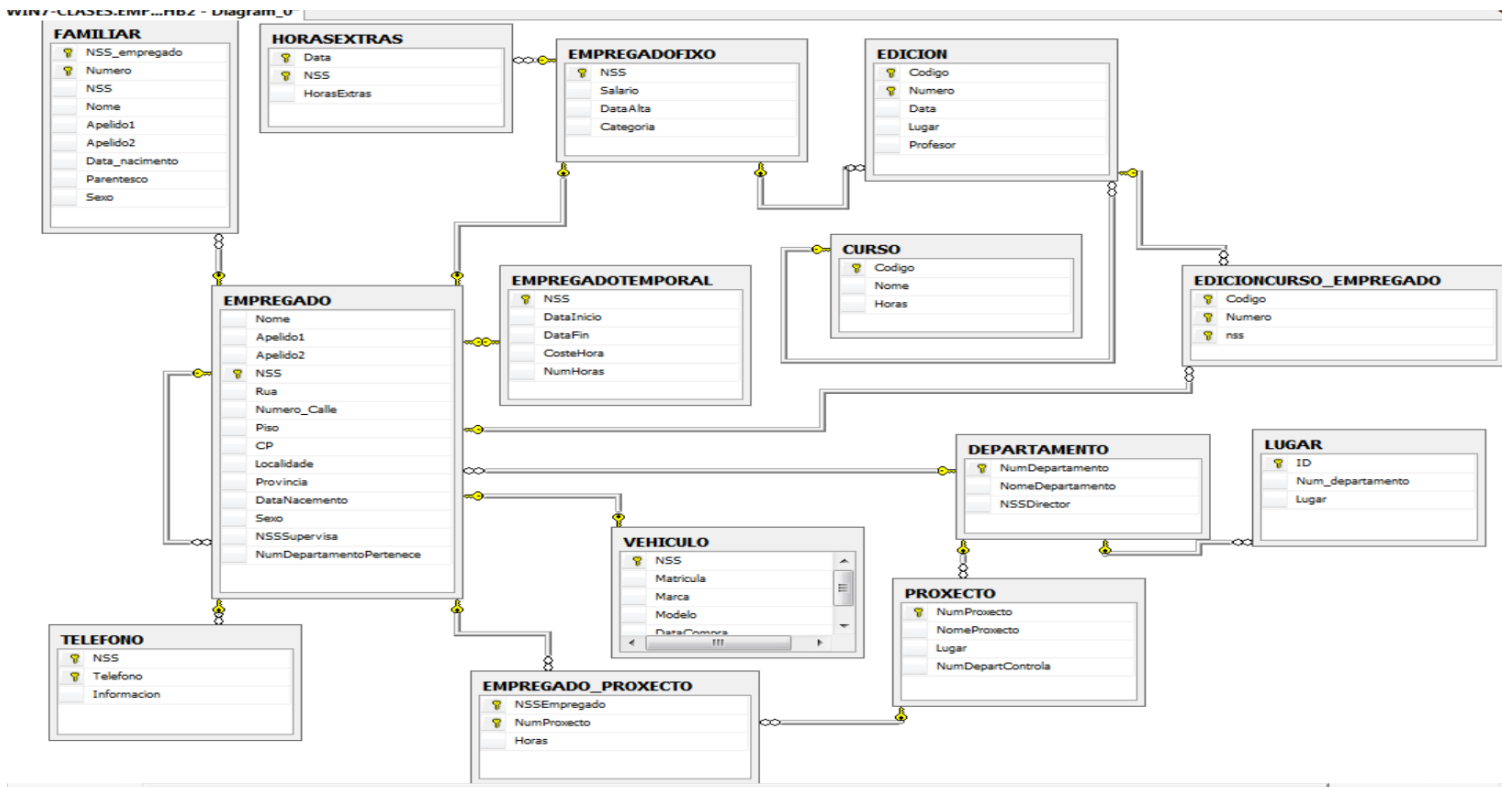
IES CHAN DO MONTE

C.S. de Desarrollo de Aplicaciones Multiplataforma

Modulo Acceso a datos

Actividad 1 Tarea 1. Ejercicio completo de mapeo

Vamos a mapear y trabajar con una base de datos EmpresaHB26, ya creada en SQL Server y con datos para utilizarla. La estructura de la base de datos es la siguiente



Se proporcionan **dos proyectos Java independientes**:

- **EMPRESAHB26ALU_XML** → mapeo con **ficheros .hbm.xml**
- **EMPRESAHB26ALU_ANNOT** → mapeo con **anotaciones Hibernate/JPA**

En ambos proyectos:

- Inicialmente, los POJOs **solo tienen propiedades simples**.
- Las entidades tienen el mapeo del atributo identificador y la estrategia de generación de clave.
- Pojos que son componentes o colecciones de componentes no están mapeadas
- Las relaciones entre entidades no están mapeadas aún.
- Se trabajará progresivamente para convertir claves foráneas en relaciones.

Ejercicio 1: Comprender cómo funciona Hibernate

- Analiza el fichero `hibernate.cfg.xml`, donde se define la **configuración de conexión a la base de datos**, las **propiedades de Hibernate** y los **ficheros de mapeo utilizados**, o los **POJOs con anotaciones** que representan las entidades del proyecto. Observa cómo estos elementos permiten que Hibernate gestione la persistencia de datos.
- Analiza la **estructura de las clases (POJOs)** que representan la información de la base de datos. Identifica qué **especificaciones deben cumplir** para poder ser gestionadas por Hibernate.

Nota: Por ahora no se han definido las relaciones entre entidades ni las clases componentes.

- C. Analiza la estructura de los ficheros de mapeo (.hbm.xml) y también las anotaciones principales si se está usando mapeo con anotaciones. Observa cómo se definen:
- La correspondencia entre las propiedades de las clases y las columnas de las tablas de la base de datos.
 - Los **tipos de datos Hibernate** usados y si las columnas permiten o no valores nulos.
 - La **propiedad identificadora** de cada entidad y su **estrategia de generación** (*assigned* → asignada por el usuario, *increment* → generada por Hibernate o *identity* → la genera el SGBD, usualmente con **IDENTITY**, **AUTO_INCREMENT**, **SEQUENCE**).
- D. Clase HibernateUtil y SessionFactory
- Analiza cómo se inicializa la SessionFactory y su papel en la gestión de sesiones.
- E. Sesiones y contexto de persistencia
- Cómo **abrir y cerrar sesiones**.
 - Gestión de **transacciones**.
 - Analiza el **contexto de persistencia** asociado a una sesión y los **estados de los objetos**: Transient, Persistent, Detached y Removed.
 - Operaciones **CRUD** sobre objetos: save, saveOrUpdate, update, delete.
 - crear un método que reciba un curso y lo inserte en la base de datos.
- F. Acceso a datos por ID
- Dado un número de proyecto, visualizar sus datos.
 - Mostrar un **mensaje de error** si no existe el proyecto.

Ejercicio 2: MAPEO DE COMPONENTES y realizacion de operaciones CRUD

Ten en cuenta que la claves del departamento y de proyecto serán generadas por Hibernate y la clave de curso es generada por SQL Server.

A. Dirección del empleado como componente:

La dirección del empleado (calle, número, CP, localidad y provincia) se representará mediante una **clase independiente** y se mapeará como **componente** dentro del POJO Empleado.

B. Funciones del departamento:

Se quiere ampliar el modelo de datos de la empresa para permitir registrar las distintas **funciones o responsabilidades** que desempeña cada departamento dentro de la empresa. Cada departamento podrá tener **una o varias funciones**, como por ejemplo: “Selección de personal”, “Gestión de nóminas”, etc.

1. Crear la estructura necesaria en la base de datos
Diseña una tabla que permita almacenar múltiples funciones asociadas a un mismo departamento.
2. Añadir la colección como SET al POJO Departamento (es tipo valor o componente).
3. Mapear la colección: Realiza el mapeo de esta colección de funciones
4. Implementa un método que permita insertar las funciones de un departamento dado. El método recibirá el numdepartamento y un set de funciones y se insertarán en la tabla correspondiente.
5. Implementa un método que permita eliminar una función concreta asociada a un departamento. Mostrar un mensaje adecuado si la función no existe o no pertenece al departamento.

C. Fases del proyecto.

Se quiere ampliar el modelo de datos de la empresa para permitir registrar las distintas fases por las que pasa un proyecto durante su desarrollo.

Cada proyecto podrá tener una o varias fases, identificadas por un nombre único dentro del propio proyecto, y cada fase tendrá además un estado, como por ejemplo: “Análisis” – “En proceso”, “Diseño” – “Pendiente”

1. Crear la estructura necesaria en la base de datos
Diseña y crea una tabla que permita almacenar múltiples fases asociadas a un mismo proyecto. Se añadirá un atributo que indique el estado de la fase.

2. Añadir la colección con SET de componentes al POJO Proyecto
3. Mapear la colección: Realiza el mapeo de esta colección de fases.
4. Implementa un método que permita añadir una o varias fases nuevas a un proyecto existente. Mostrar un mensaje adecuado si el proyecto no existe.
5. Implementa un método que reciba: un numero de un proyecto, el nombre de una fase y un estado y realice la siguiente operación:
 - a. Si la fase **ya existe** dentro del proyecto, deberá **actualizarse su estado**.
 - b. Si la fase **no existe**, deberá **insertarse una nueva fase** con el nombre y el estado indicados.

El método deberá persistir los cambios mediante Hibernate y mostrar un mensaje adecuado indicando si la fase fue actualizada o insertada. Mostrar un mensaje adecuado si el proyecto no existe.

D. Mapeo de los teléfonos de los empleados

Los teléfonos de los empleados se representarán como una colección MAP de componentes (tipo valor), donde:

- La clave del MAP el numero del telefono
 - y el valor será el tipo de teléfono (por ejemplo: “casa”, “móvil”, “trabajo”),
- Se deberá:

1. Añadir el MAP al POJO Empleado.
2. Realizar el mapeo
3. Implementa un método que reciba: el nss del empleado, un número de teléfono y un tipo de teléfono (por ejemplo: “casa”, “móvil”, “trabajo”) y realice la siguiente operación:
 - a. Si el número de teléfono ya existe dentro del MAP de teléfonos del empleado, deberá actualizarse su tipo.
 - b. Si el número de teléfono no existe, deberá **insertarse una nueva entrada** en el MAP, usando el número como clave y el tipo como valor.

El método deberá persistir los cambios mediante Hibernate y mostrar un mensaje adecuado indicando si el teléfono fue actualizado o insertado. Si el empleado no existe, deberá mostrarse un mensaje apropiado.

4. Implementa un método para **borrar un teléfono** dun empleado dado o seu NSS e o número de teléfono.
 - Se o empregado non existe → mensaxe correspondente.
 - Se o teléfono non existe → mensaxe correspondente.
 - Se existe → borrar e confirmar a operación.

E. Mapeo de los familiares de los empleados

Cada empleado puede tener varios familiares, almacenados en una lista de componentes. la lista deberá mantener un orden estable, y cada familiar tendrá un índice asociado en la tabla correspondiente.

- En el mapeo XML, Hibernate permite controlar el índice mediante la etiqueta <list-index>, pudiendo comenzar en 1 si así lo requiere la estructura de la base de datos.
- En el mapeo con anotaciones, Hibernate no permite establecer un índice inicial distinto de 0, Por defecto, la anotación: @ OrderColumn(name = "Numero") hará que el primer elemento tenga índice 0. Si la base de datos ya está diseñada para comenzar en 1, será el usuario quien deberá garantizar la coherencia, ajustando manualmente los índices para que coincida con la estructura existente.

1. Insertar un familiar en un empleado

Desarrolla un método que reciba: El NSS del empleado y los datos de un Familiar.

El método deberá:

- Comprobar si el empleado existe. Si no existe → mostrar mensaje adecuado

- Comprobar si ya existe un familiar con el mismo NSS dentro del empleado. Si existe → no insertar y mostrar mensaje.
- Si no existe → insertar el familiar en la lista y mostrar un mensaje indicando que el familiar fue insertado correctamente

2. Buscar un familiar dentro de un empleado .

Desarrolla un método que reciba: El NSS del empleado y el NSS del familiar

El método deberá:

- Si el empleado no existe → mostrar mensaje adecuado
- Si el familiar no existe → mostrar mensaje adecuado
- Si existe → mostrar sus datos

Este método puede implementarse tanto con **búsqueda en memoria** (recorriendo la lista) como con **HQL** mediante una consulta (Permite delegar el filtrado a la base de datos y evita cargar colecciones completas en memoria). Realízalo con una consulta HQL trayendo solo el familiar dado si existe. Si no existe el familiar mensaje de error.

Ejercicio 3: MAPEO DE ASOCIACIONES y realización de operaciones CRUD

En este ejercicio se trabajará el mapeo de asociaciones entre clases del modelo de datos utilizando Hibernate. El objetivo es comprender cómo representar relaciones entre objetos Java y tablas de la base de datos, aplicando correctamente los distintos tipos de asociaciones que pueden existir en un sistema empresarial. Deberás analizar cada relación propuesta y realizar su **mapeo en Hibernate**, utilizando XML y anotaciones según se indique en cada apartado.

Las asociaciones entre entidades se realizarán **BIDIRECCIONAL**. Usa la característica de persistencia transitiva pues que más se adecue en cada caso.

Tenemos las siguientes Entidades y asociaciones

1.- Empleados y departamentos:

- La empresa almacena información sobre sus empleados. Cada empleado **pertenece a un único departamento**, mientras que **cada departamento puede tener varios empleados asociados**.
- Los empleados de un departamento se representarán mediante una **colección de tipo SET**

A. Realizar el mapeo

Mapear la asociación bidireccional entre empleados y departamentos, indicando la clave foránea correspondiente y la colección SET en el lado del departamento.

B. Insertar un empleado junto al departamento donde está

Insertar un nuevo empleado junto con el departamento al que pertenece, asegurando que la relación queda correctamente establecida en ambos sentidos.

El método recibe un objeto empleado y el nombre de un departamento, y lo inserta en la base de datos. Visualizar mensajes de errores

C. Modificar el departamento de un empleado

Dado un número de departamento y un nss de empleado, se le cambia de departamento.

2.- Proyectos y departamentos:

- ✓ La empresa gestiona distintos proyectos, cada uno de los cuales tiene un nombre único y está controlado por un único departamento. Un departamento puede controlar varios proyectos de forma simultánea.
- ✓ Los proyectos que controla un departamento se representarán mediante una colección de tipo BAG en el lado del departamento.

A. Realizar el mapeo

Mapear la asociación bidireccional entre empleados y departamentos, indicando la clave foránea correspondiente y la colección SET en el lado del departamento.

Definir adecuadamente el lado propietario de la relación.

B. Listado de proyectos que controla un departamento

Dado el nombre de un departamento, listar todos los proyectos que controla dicho departamento.

✓ C. Modificar el departamento que controla un proyecto

Dado el nombre de un proyecto y el nombre de un nuevo departamento, modificar el departamento que pasa a controlar dicho proyecto, asegurando que la relación queda correctamente actualizada en ambos sentidos.

3.- Empleados y Habilidades Profesionales

La empresa desea ampliar el modelo de datos para gestionar las habilidades profesionales de sus empleados.

Cada empleado puede poseer varias habilidades, y cada habilidad puede estar asociada a múltiples empleados.

Cada habilidad se identifica de forma única por su código autonumérico y se guarda un nombre único.

A.-Crea las estructuras necesarias en la base de datos

B.- Realizar el mapeo

Implementar el mapeo entre empleados y habilidades en Hibernate, cumpliendo los siguientes requisitos:

1. La relación debe ser **bidireccional**.
2. En la clase **Empleado**, se incluirá una colección de habilidades representada mediante un **SET**.
3. En la clase **Habilidad**, se incluirá una colección de empleados representada también mediante un **SET**.

C.-Alta de habilidades

Haz un método que reciba una lista de objetos Habilidad (por ejemplo: "Java", "Hibernate", "Contaplus") y los inserte en la base de datos

Nota: Ten en cuenta que el campo nome en la base de datos es único; el método debe gestionar la persistencia de forma que, si el objeto es nuevo, se genere su ID autonumérico.

D.- Asignación de habilidades a un empleado

Implementa un método que permita vincular habilidades a un empleado existente

El método recibe nss y una lista de habilidades y vincule estas habilidades al empleado.

Lógica: Deberás recuperar al empleado, añadir las habilidades a su colección Set y actualizar el estado de la entidad. Observa cómo Hibernate gestiona automáticamente la inserción en la tabla intermedia.

E.- Borrado de una habilidad

La empresa desea eliminar del sistema aquellas habilidades que han quedado obsoletas.

Implementa un método que reciba el ID de una habilidad y la elimine de la base de datos.

Nota: Borrar una habilidad no debe borrar al empleado, pero sí se debe borrar la tabla intermedia. Cuidado con lo que se mapea en cascada

4.-Vehiculo y Empleado

La empresa desea gestionar la información de los vehículos asignados a sus empleados. Cada empleado puede tener **como máximo un vehículo**, y cada vehículo pertenece **a un único empleado**.

Para garantizar esta restricción, la tabla **VEHICULO** utilizará como clave primaria el mismo valor que la clave primaria del empleado (NSS). Es decir, **la clave primaria del vehículo será también una clave foránea que referencia a EMPREGADO**.

Un vehículo **no puede existir sin un empleado asociado**.

Un empleado **solo puede tener un vehículo**, ya que la PK del vehículo es única y coincide con la del empleado.

A.- Realizar el mapeo

- La relación entre **Empleado** y **Vehiculo** será **One-to-One**.
- La clave primaria de **Vehiculo** será la misma que la de **Empleado**.

- La relación será **bidireccional**:
 Empleado tendrá un atributo vehiculo.
 Vehiculo tendrá un atributo empleado.

B.- Insertar o actualizar el vehículo de un empleado

Realiza un método que dado un nss, registrar o modificar su vehículo. La matrícula es única.
 Controla los errores.

C.- Eliminar el vehículo asociado a un empleado

La empresa desea permitir que un empleado pueda **dar de baja su vehículo**.

Implementa un método que dado un nss permita eliminar el registro de la tabla VEHICULO cuyo NSS coincide con el del empleado.

5.- Proyectos y empleados

La empresa desea gestionar la participación de los empleados en los distintos proyectos que desarrolla.

Un empleado puede participar en varios proyectos, y un proyecto puede tener asignados varios empleados.

La relación entre empleados y proyectos incluye un dato adicional: **el número de horas** que cada empleado dedica a cada proyecto.

A. Realizar el mapeo

La relación entre **Empleado** y **Proyecto** no es una relación *Many-to-Many pura*, ya que la tabla intermedia **almacena un atributo adicional**: el número de horas que cada empleado dedica a cada proyecto. Por este motivo, la relación debe modelarse mediante una **entidad intermedia propia (un pojo)**, que representará la tabla **EMPREGADO_PROXECTO**.

Se utilizará Set para las colecciones y será bidireccional

- **Entidad intermedia**: La relación se modelará mediante una entidad (POJO) intermedia, por ejemplo EmpleadoProyecto, que representará la tabla EMPREGADO_PROXECTO.
 Esta entidad contendrá:
 - Una clave primaria compuesta, formada por: nss (empleado) numProyecto (proyecto).
 La clave primaria compuesta se mapeará mediante una clase Java adicional, que actuará como componente embebido (@Embeddable)
 - Una referencia a Empleado
 - Una referencia a **Proyecto**
 - El atributo **horas**

• Colecciones en los extremos de la relación

En la clase **Empleado**, se incluirá una colección **SET** de participaciones: Set<EmpleadoProyecto>

En la clase **Proyecto**, se incluirá también un **SET** de participaciones: Set<EmpleadoProyecto>

La relación será **bidireccional**, permitiendo navegar desde el empleado hacia sus proyectos y desde el proyecto hacia sus empleados.

B. Listado de proyectos y horas por empleado

Desarrollar un método que, para **cada empleado**, muestre: NSS, Nombre y apellidos del empleado y los proyectos donde participe :Nombre del proyecto y Número de horas dedicadas

Si un empleado no participa en ningún proyecto, deberá indicarse expresamente.

a) Versión usando un constructor expression en HQL

La consulta HQL debe crear directamente objetos de un DTO llamado EmpleadoProyectoDTO.

El DTO contendrá los campos necesarios para mostrar la información.

La capa de lógica recorrerá la lista de DTOs y mostrará los datos.

✓ ¿Qué es un constructor expresión?

Un **constructor expresión** sirve para que una consulta HQL **crea directamente objetos de una clase personalizada (DTO)**. Permite que Hibernate construya objetos a medida directamente desde la consulta, usando solo los campos que tú necesitas.

✓ Sintaxis general

```
select new paquete.NombreDTO(campo1, campo2, campo3)
from Entidad e where ...
```

b) Versión usando List<Object[]>

- La consulta HQL solo traerá los campos necesarios mediante un SELECT parcial.
- La capa de lógica mostrará los datos recorriendo la lista de arrays.

la consulta HQL **no devuelve entidades completas**, sino **solo los campos necesarios**, empaquetados en un array de objetos (Object[]). La capa de lógica será la encargada de recorrer ese array y mostrar los datos.

¿Qué es una consulta HQL que devuelve List<Object[]>?

Es una consulta HQL que utiliza un **SELECT parcial**, es decir, selecciona únicamente los campos que necesitas, no la entidad completa.

Hibernate devuelve cada fila como un **Object[]**, donde:

- fila[0] → primer campo del SELECT
- fila[1] → segundo campo
- fila[2] → tercero y así sucesivamente

Es muy eficiente porque **no carga objetos completos**, solo los valores necesarios.

Sintaxis general

```
String hql = ""
select campo1, campo2, campo3
from Entidad e
where ... "";
```

```
List<Object[]> lista = session.createQuery(hql, Object[].class).list();
```

¿Qué contiene cada Object[]?

Cada fila del resultado contiene:

```
fila[0] → campo1
fila[1] → campo2
fila[2] → campo3
```

C. Borrado de un proyecto

Desarrollar un método que, dado el **nombre de un proyecto**, lo elimine de la base de datos.

6.- Supervisión entre empleados

La empresa desea registrar la relación de supervisión entre empleados.

Un empleado puede supervisar a varios empleados. Cada empleado puede tener como máximo un supervisor.

Los empleados supervisados por un empleado se representarán mediante una colección de tipo SET.

A. Realizar el mapeo

Mapear la asociación **bidireccional de supervisión** entre empleados utilizando Hibernate.

Representar los empleados supervisados mediante una **colección SET** en el lado del supervisor. Indicar claramente el **lado propietario de la relación**.

B. Quitar supervisor a un empleado

Dado el NSS de un empleado, se desea **desasignar su supervisor**.

El método deberá cumplir las siguientes condiciones:

- Si el empleado **no existe**, mostrar un mensaje indicando que no se encontró.
- Si el empleado **no tiene supervisor asignado**, mostrar un mensaje indicando que no tiene supervisor.
- Si el empleado **tiene un supervisor**, mostrar primero los datos del supervisor actual (nombre y apellidos) y luego desasignarlo (valor null).

C.- Consulta de datos empleados y datos supervisor

Dado el NSS de un empleado, visualice nombre y apellidos del empleado y nombre y apellidos de su supervisor (si lo tiene).

El método deberá cumplir las siguientes condiciones:

1. **Si el empleado no existe**, deberá mostrarse un mensaje adecuado.
2. **Si el empleado existe pero no tiene supervisor**, los datos del supervisor deberán mostrarse **en blanco**, sin provocar errores.
3. **Si el empleado tiene supervisor**, deberán mostrarse correctamente su nombre y apellidos.
4. No se deben cargar colecciones innecesarias; basta con recuperar el empleado y su supervisor.

7.- Mapeo de la herencia entre empleados fijos y temporales y relación con el director de un departamento

La empresa desea modelar dos tipos diferentes de empleados: **empleados fijos** y **empleados temporales**. Ambos comparten una serie de atributos comunes (NSS, nombre, apellidos, fecha de nacimiento, sexo, dirección, etc) que se almacenan en la clase **Empleado**. A partir de esta clase base se crean dos clases que heredan:

- **EmpleadoFijo:** NSS (PK), dataAlta (LocalDate), salario (Double) y categoria (String, 20 caracteres)
- **EmpleadoTemporal:** NSS (PK y FK a Empleado), dataInicio (LocalDate), dataFin (LocalDate), costeHora (Double), numHoras (Double)

A. Realizar el mapeo

La herencia debe mapearse en Hibernate utilizando la estrategia **JOINED** (ya que existen 3 tablas: tabla EMPREGADO con los datos comunes, EMPREGADOFIXO con los datos específicos de los empleados fijos, EMPREGADOTEMPORAL con los datos específicos de los empleados temporales)

La clave primaria de las tablas EMPREGADOFIXO y EMPREGADOTEMPORAL es también una **clave foránea** que referencia a la tabla EMPREGADO.

B.- Relación entre el director de un departamento y Empleado Fijo

Cada departamento tiene asignado un **director**, que siempre será un **empleado fijo**. La empresa establece las siguientes restricciones funcionales:

- Un departamento **solo puede tener un director**.
- Un empleado fijo **solo puede ser director de un único departamento**.

Para garantizar estas restricciones en la base de datos, es necesario completar la definición de la tabla **DEPARTAMENTO** añadiendo:

1. Restricción UNIQUE sobre la columna NSSDirector

Esta restricción asegura que un mismo empleado fijo no pueda aparecer como director en más de un departamento, garantizando así la relación entre Departamento y EmpleadoFijo.

```
ALTER TABLE DEPARTAMENTO
```

```
ADD CONSTRAINT UQ_DepartamentoDirector UNIQUE (NSSDirector);
```

2. Clave foránea hacia la tabla EMPREGADOFIXO

La columna NSSDirector debe ser también una clave foránea que referencie al NSS del empleado fijo que actúa como director. Esta relación no estaba definida en la base de datos y debe añadirse:

```
ALTER TABLE DEPARTAMENTO
```

```
ADD CONSTRAINT FK_DEPARTAMENTO_EMPREGADOFIXO FOREIGN KEY (NSSDirector)
REFERENCES EMPREGADOFIXO (NSS);
```

Con estas dos restricciones:

- Se garantiza que **cada departamento tiene un único director**.
- Se garantiza que **cada empleado fijo solo puede dirigir un departamento**.
- Se establece correctamente la integridad referencial entre DEPARTAMENTO y EMPREGADOFIXO.

✓ **Lado propietario (Departamento):**

El lado propietario de la relación es **Departamento**, ya que contiene la clave foránea NSSDirector. El mapeo se realiza mediante un <many-to-one> con unique="true" para reflejar la relación 1:1:

¿Por qué NO se puede poner <one-to-one> en este lado?

Porque **Hibernate no decide la cardinalidad por la lógica del modelo, sino por dónde está la clave foránea (FK) en la base de datos.**

- El lado que tiene la FK siempre se mapea con <many-to-one>. Aunque la relación sea 1:1 real.
- El lado que NO tiene la FK se mapea con <one-to-one>: Es el lado inverso.

✓ **Lado inverso (EmpleadoFijo):** Se puede mapearse la relación inversa mediante un <one-to-one>:

3.- Listado de empleados con tipo de empleado

Crea un método llamado que recupere un listado de empleados utilizando **HQL**.

Se debe visualizar:

- NSS del empleado,
- **Nombre Completo:** Concatenación de nombre y apellidos (gestionando posibles nulos en el segundo apellido),
- **Tipo de Empleado:** Mediante una expresión CASE, debe devolver el texto "Empleado Fijo" si la entidad es de tipo EmpleadoFijo y "Empleado Temporal" si es EmpleadoTemporal.
- **Departamento:** Si el empleado es director de un departamento, mostrar el nombre del departamento; si no lo es, mostrar un texto vacío o "---".

El listado debe venir ordenado alfabéticamente por el nombre del empleado.

Nota: La función **type(e)** identifica dinámicamente a qué subclase pertenece la entidad en el momento de la consulta.

Ejemplo: when type(e) = POJOS.EmpleadoFijo then 'Fijo'.

LISTADO DE EMPREGADOS

```
-----
NSS: 8888889 | Nome: Agostiño Cerviño Seoane | Tipo: Empleado Fijo | Departamento Dirixido: INFORMÁTICA
NSS: 7000007 | Nome: Alex Méndez García | Tipo: Empleado Temporal | Departamento Dirixido:
NSS: 00000080 | Nome: Ana Souto Santiago | Tipo: null | Departamento Dirixido:
NSS: 7777777 | Nome: Ana María Ramilo Barreiro | Tipo: Empleado Fijo | Departamento Dirixido: INNOVACIÓN
NSS: 9876567 | Nome: Ángeles López Arias | Tipo: Empleado Temporal | Departamento Dirixido:
NSS: 4444999 | Nome: Antonia Romero Boo | Tipo: Empleado Temporal | Departamento Dirixido:
NSS: 5555000 | Nome: Anxos Loures Freire | Tipo: Empleado Temporal | Departamento Dirixido:
NSS: 6000006 | Nome: Beatriz Mallo López | Tipo: Empleado Fijo | Departamento Dirixido:
NSS: 9900000 | Nome: Breixo Pereiro Lamela | Tipo: Empleado Fijo | Departamento Dirixido:
NSS: 6000600 | Nome: Carme Jurado Vega | Tipo: Empleado Temporal | Departamento Dirixido:
NSS: 6666666 | Nome: Galia Bruno Valiña | Tipo: Empleado Temporal | Departamento Dirixido:
```

4.- Inserción de empleados (con tipo) y departamento

Crea un método que permita **insertar un empleado (fijo o temporal)** en la base de datos indicando:

- Los datos personales del empleado (NSS, nombre, apellidos, fecha de nacimiento, sexo y dirección) y según el tipo de empleado que podrá ser:
 - **EmpleadoFijo**, incluyendo salario, fecha de alta y categoría.
 - **EmpleadoTemporal**, incluyendo fecha de inicio, fecha de fin, coste por hora y número de horas.
- El nombre del departamento al que pertenece el empleado.
- Devolver un código indicando el resultado de la operación:
 - 0 → Inserción correcta
 - 1 → NSS ya existente
 - 2 → Departamento no encontrado
 - 4 → Error genérico

REPASO

8.- Mapeo de las horas extras que hace un empleado.

Se desea llevar un control de las horas extras realizadas por los empleados Fijos de la empresa.

Cada empleado fijo puede realizar cero, una o varias jornadas de horas extras, y cada jornada se identifica por:

- La fecha en la que se realizaron las horas
- El NSS del empleado al que pertenecen
- El número de horas extras realizadas ese día

A. Realizar el mapeo

Esta información se debe mapear en Hibernate como una **colección Map de componentes** asociado a un empleado fijo, de forma que cada empleado tenga un conjunto de registros de horas extras asociadas a distintas fechas. La clave primaria de la tabla es (**NSS, Data**), por lo que un empleado solo puede tener **un registro por día**.

B.-Insertar o modificar las horas extras de un empleado

Un método que permita añadir o modificar las horas extras de un empleado fijo.

- Si el empleado ya tiene horas extras registradas para esa fecha, deben **actualizarse**.
- Si no existen, deben **insertarse**.

Se deben controlar los errores imprimiendo un mensaje al respecto

C.-Eliminar el registro de horas extras de un empleado en una fecha

un método que reciba como parámetros: NSS del empleado y la fecha de las horas extras y elimine este registro en la base de datos

Controlar las posibles situaciones de error: empleado inexistente, fecha sin horas registradas.

D.- Visualizar el total de horas extras de un empleado ha hecho en un mes de un año dado.

Crear un método que, dado un NSS, un mes y un año, devuelva la suma total de horas extras realizadas por dicho empleado en ese periodo mediante una consulta **HQL**.

Nota : El uso de Map en HQL

- Cuando mapeas una colección como un `<map>`, Hibernate almacena los datos como pares de **Clave-Valor**. En HQL, al no ser una entidad con atributos propios, debes usar funciones especiales para acceder a los datos:

Acceso a los datos en la consulta:

Para una entrada del mapa que definamos como `join e.horasextras h`:

- **La Clave (Key):** Representa el índice del mapa (en este caso, la **Fecha**).
 - Puedes usar indistintamente **key(h)** o **index(h)**. Ambos devuelven el valor de la columna definida en `<map-key>`.
- **El Valor (Value):** Representa el contenido asociado a esa clave (en este caso, el **Número de Horas**).
 - Puedes usar **value(h)** o **elements(h)**. Devuelven el valor de la columna definida en `<element>`.

Ejemplo :

```
SELECT SUM(value(h)) //(Suma las horas, que es el valor)
FROM Empregadofixo e JOIN e.horasextras h
WHERE key(h)=fecha // (Filtra por la fecha, que es la clave del mapa)
```

9.- Mapeo de los lugares de un departamento

Se desea llevar un control de las distintas ubicaciones en las que un **Departamento** desarrolla su actividad. Cada departamento puede estar ubicado en uno o varios lugares físicos.

A. Realizar el mapeo

Lugar se mapea como entidad, teniendo en cuenta que **ID del lugar lo genera Hibernate**.

Un **departamento** puede tener **varios lugares** asociados.

Un **mismo nombre de lugar** puede aparecer en varios departamentos, pero **cada fila de la tabla LUGAR pertenece a un único departamento**.

Existe una **restricción de unicidad** que impide que un mismo departamento tenga duplicado el mismo nombre de lugar.

Los lugares de los departamentos se mapean como colección set

La clave ajena **Num_departamento** está en la tabla LUGAR, por lo que la relación es **Many-to-One** desde Lugar hacia Departamento.

NOTA: Aunque un mismo nombre de lugar pueda aparecer en varios departamentos, **NO se trata de una relación Many-to-Many**, ya que para ello sería necesaria una tabla intermedia (por ejemplo, DEPARTAMENTO_LUGAR que contenga los IDs de ambas).

La tabla LUGAR actúa como **entidad propia**, no como tabla intermedia.

B.-Gestión de los lugares de un Departamento

Realizar un método que, dado un nombre de departamento, visualice los lugares donde se encuentra

C.-Inserción de un departamento , los lugares , la funciones y el director

El método recibirá un nuevo departamento, una lista de lugares, una lista de funciones y el nss del director y lo inserte en la BD.

Controlar los posibles errores.